

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Kavana M A (1BM23CS145)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Jan-2026 to June-2026

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Machine Learning (23CS6PCMAL)” carried out by **Kavana M A (1BM23CS145)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Saritha A N Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
--	--

Index

Sl. No.	Date	Experiment Title	Page No.
1	12-2-2026	Write a python program to import and export data using Pandas library functions	1 – 4
2	29-1-2026	Demonstrate various data pre-processing techniques for a given dataset	5 – 8
3	26-2-2026	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	9 – 12
4	26-2-2026	Build Logistic Regression Model for a given dataset	13 – 16
5	26-2-2026	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	17 – 21
6	05-3-2026	Build KNN Classification model for a given dataset.	22 – 25
7	12-3-2026	Build Support vector machine model for a given dataset	26 – 28
8	26-3-2026	Implement Random forest ensemble method on a given dataset.	29 – 30
9	26-3-2026	Implement Boosting ensemble method on a given dataset.	31 – 32
10	02-4-2026	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	33 – 34
11	09-4-2026	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	35 – 37

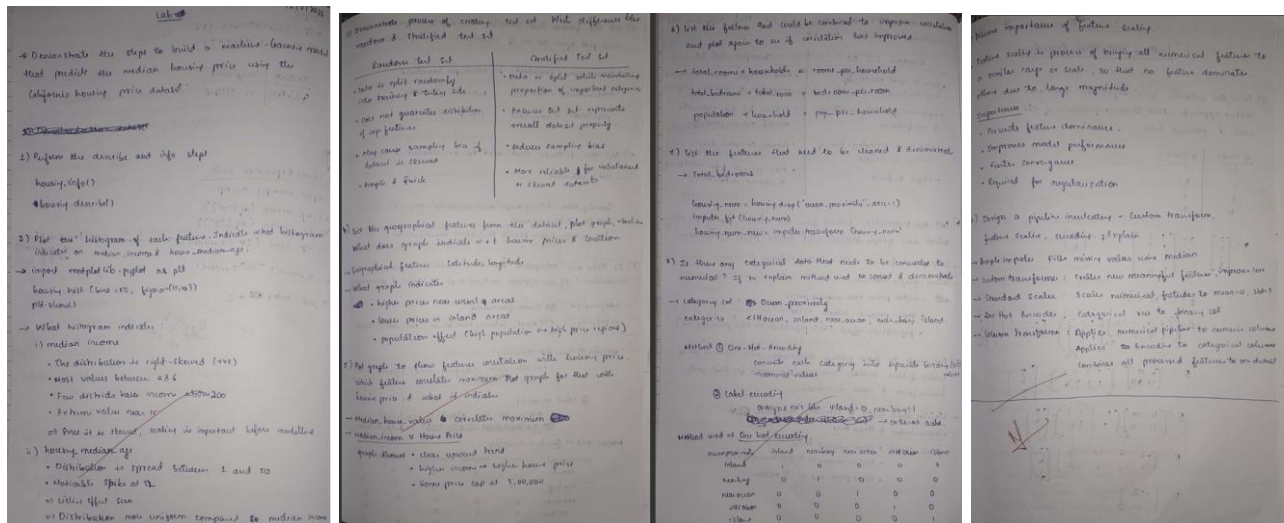
Github Link:

https://github.com/kavana-ma/ML_sem6

Program 1

Write a python program to import and export data using Pandas library functions

Screenshot



Code:

```
import pandas as pd
# Load dataset
housing = pd.read_csv("housing.csv")
# Display first 5 rows
housing.head()
housing.info()
housing.describe()
import matplotlib.pyplot as plt

housing.hist(bins=50, figsize=(15,10))
plt.show()

from sklearn.model_selection import train_test_split

train_set, test_set = train_test_split(housing, test_size=0.2, random_state=42)

import numpy as np

housing["income_cat"] = pd.cut(housing["median_income"],
                               bins=[0., 1.5, 3.0, 4.5, 6., np.inf],
```

```

labels=[1,2,3,4,5])

from sklearn.model_selection import StratifiedShuffleSplit

split = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)

for train_index, test_index in split.split(housing, housing["income_cat"]):
    strat_train_set = housing.loc[train_index]
    strat_test_set = housing.loc[test_index]

housing.plot(kind="scatter", x="longitude", y="latitude",
              alpha=0.4,
              s=housing["population"]/100,
              label="population",
              c="median_house_value",
              cmap="jet",
              colorbar=True)
plt.legend()
plt.show()

corr_matrix = housing.corr(numeric_only=True)

corr_matrix["median_house_value"].sort_values(ascending=False)

housing.plot(kind="scatter", x="median_income", y="median_house_value", alpha=0.1)
plt.show()

housing["rooms_per_household"] = housing["total_rooms"] / housing["households"]
housing["bedrooms_per_room"] = housing["total_bedrooms"] / housing["total_rooms"]
housing["population_per_household"] = housing["population"] / housing["households"]

corr_matrix = housing.corr(numeric_only=True)
corr_matrix["median_house_value"].sort_values(ascending=False)

housing["total_bedrooms"].isnull().sum()
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")

housing_num = housing.drop("ocean_proximity", axis=1)

imputer.fit(housing_num)

housing_num_imputed = imputer.transform(housing_num)

housing["ocean_proximity"].value_counts()
from sklearn.preprocessing import OneHotEncoder

```

```

housing_cat = housing[["ocean_proximity"]]

encoder = OneHotEncoder()

housing_cat_encoded = encoder.fit_transform(housing_cat)

housing_cat_encoded.toarray()

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

housing_scaled = scaler.fit_transform(housing_num_imputed)

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.base import BaseEstimator, TransformerMixin

# Custom Transformer
class CombinedAttributesAdder(BaseEstimator, TransformerMixin):
    def __init__(self, add_bedrooms_per_room=True):
        self.add_bedrooms_per_room = add_bedrooms_per_room

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        rooms_per_household = X[:, 3] / X[:, 6]
        population_per_household = X[:, 5] / X[:, 6]

        if self.add_bedrooms_per_room:
            bedrooms_per_room = X[:, 4] / X[:, 3]
            return np.c_[X, rooms_per_household, population_per_household, bedrooms_per_room]
        else:
            return np.c_[X, rooms_per_household, population_per_household]

num_attribs = list(housing_num.columns)
cat_attribs = ["ocean_proximity"]

num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy="median")),
    ('attribs_adder', CombinedAttributesAdder()),
    ('std_scaler', StandardScaler())
])

full_pipeline = ColumnTransformer([
    ("num", num_pipeline, num_attribs),

```

```
    ("cat", OneHotEncoder(), cat_attribs)
])

housing_prepared = full_pipeline.fit_transform(housing)

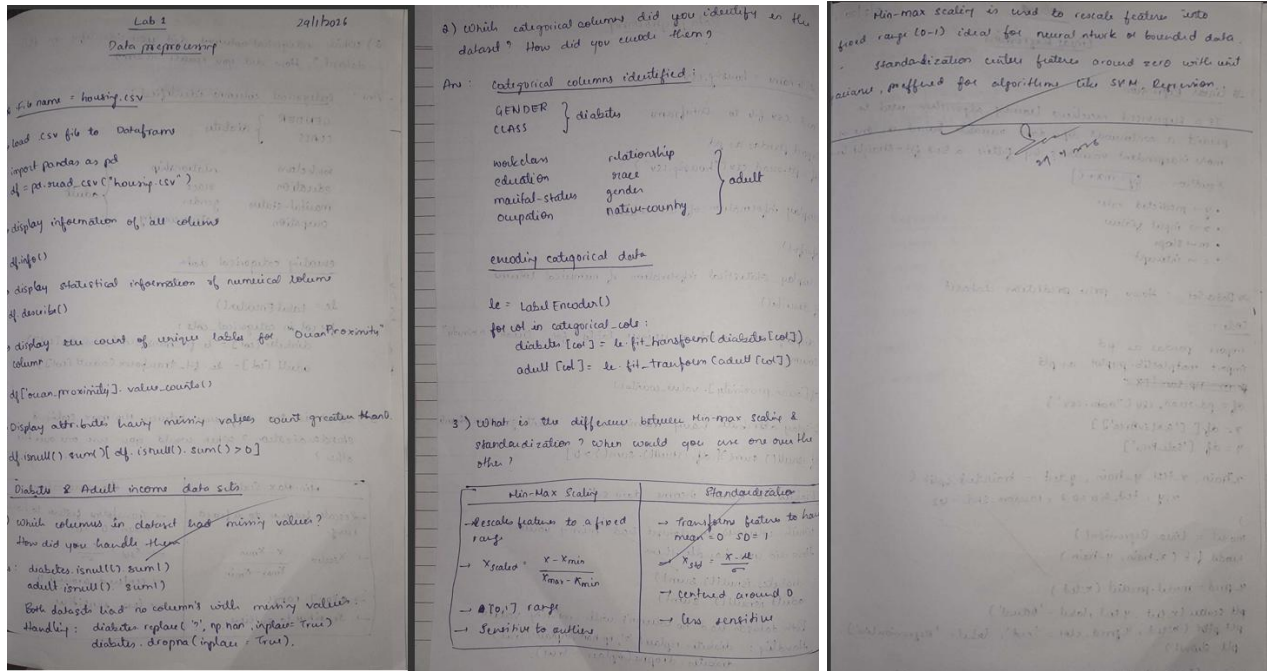
from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()
lin_reg.fit(housing_prepared, housing["median_house_value"])
```

Program 2

Demonstrate various data pre-processing techniques for a given dataset

Screenshot



Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from scipy import stats

def createdata():
    data = {
        'Age': np.random.randint(18, 70, size=20),
        'Salary': np.random.randint(30000, 120000, size=20),
        'Purchased': np.random.choice([0, 1], size=20),
        'Gender': np.random.choice(['Male', 'Female'], size=20),
        'City': np.random.choice(['New York', 'San Francisco', 'Los Angeles'], size=20)
    }

    df = pd.DataFrame(data)
    return df
df = createdata()
df.head(10)
```



```

df.shape
# Introduce some missing values for demonstration
df.loc[5, 'Age'] = np.nan
df.loc[10, 'Salary'] = np.nan
df.head(10)
# Basic information about the dataset
print(df.info())
# Summary statistics
print(df.describe())
# Code to Find Missing Values
# Check for missing values in each column
missing_values = df.isnull().sum()

# Display columns with missing values
print(missing_values[missing_values > 0])
# Set the values to some value (zero, the mean, the median, etc.).
# Step 1: Create an instance of SimpleImputer with the median strategy for Age and mean strategy
for Salary
imputer1 = SimpleImputer(strategy="median")
imputer2 = SimpleImputer(strategy="mean")

df_copy=df

# Step 2: Fit the imputer on the "Age" and "Salary" column
# Note: SimpleImputer expects a 2D array, so we reshape the column
imputer1.fit(df_copy[["Age"]])
imputer2.fit(df_copy[["Salary"]])

# Step 3: Transform (fill) the missing values in the "Age" and "Salary" column
df_copy["Age"] = imputer1.transform(df[["Age"]])
df_copy["Salary"] = imputer2.transform(df[["Salary"]])

# Verify that there are no missing values left
print(df_copy["Age"].isnull().sum())
print(df_copy["Salary"].isnull().sum())
# Handling Categorical Attributes
# Using Ordinal Encoding for gender Column and One-Hot Encoding for City Column

# Initialize OrdinalEncoder
ordinal_encoder = OrdinalEncoder(categories=[["Male", "Female"]])
# Fit and transform the data
df_copy["Gender_Encoded"] = ordinal_encoder.fit_transform(df_copy[["Gender"]])

# Initialize OneHotEncoder
onehot_encoder = OneHotEncoder()

# Fit and transform the "City" column
encoded_data = onehot_encoder.fit_transform(df[["City"]])

```

```

# Convert the sparse matrix to a dense array
encoded_array = encoded_data.toarray()

# Convert to DataFrame for better visualization
encoded_df = pd.DataFrame(encoded_array,
columns=onehot_encoder.get_feature_names_out(["City"]))
df_encoded = pd.concat([df_copy, encoded_df], axis=1)

df_encoded.drop("Gender", axis=1, inplace=True)
df_encoded.drop("City", axis=1, inplace=True)

print(df_encoded.head())
#Data Transformation
# Min-Max Scaler/Normalization (range 0-1)
#Pros: Keeps all data between 0 and 1; ideal for distance-based models.
#Cons: Can distort data distribution, especially with extreme outliers.
normalizer = MinMaxScaler()
df_encoded[['Salary']] = normalizer.fit_transform(df_encoded[['Salary']])
df_encoded.head()
# Standardization (mean=0, variance=1)
#Pros: Works well for normally distributed data; suitable for many models.
#Cons: Sensitive to outliers.
scaler = StandardScaler()
df_encoded[['Age']] = scaler.fit_transform(df_encoded[['Age']])
df_encoded.head()

#Removing Outliers
# Outlier Detection and Treatment using IQR
#Pros: Simple and effective for mild outliers.
#Cons: May overly reduce variation if there are many extreme outliers.
df_encoded_copy1=df_encoded
df_encoded_copy2=df_encoded
df_encoded_copy3=df_encoded

Q1 = df_encoded_copy1['Salary'].quantile(0.25)
Q3 = df_encoded_copy1['Salary'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
df_encoded_copy1['Salary'] = np.where(df_encoded_copy1['Salary'] > upper_bound, upper_bound,
np.where(df_encoded_copy1['Salary'] < lower_bound, lower_bound,
df_encoded_copy1['Salary']))

print(df_encoded_copy1.head())

#Removing Outliers

```

Z-score method

#Pros: Good for normally distributed data.

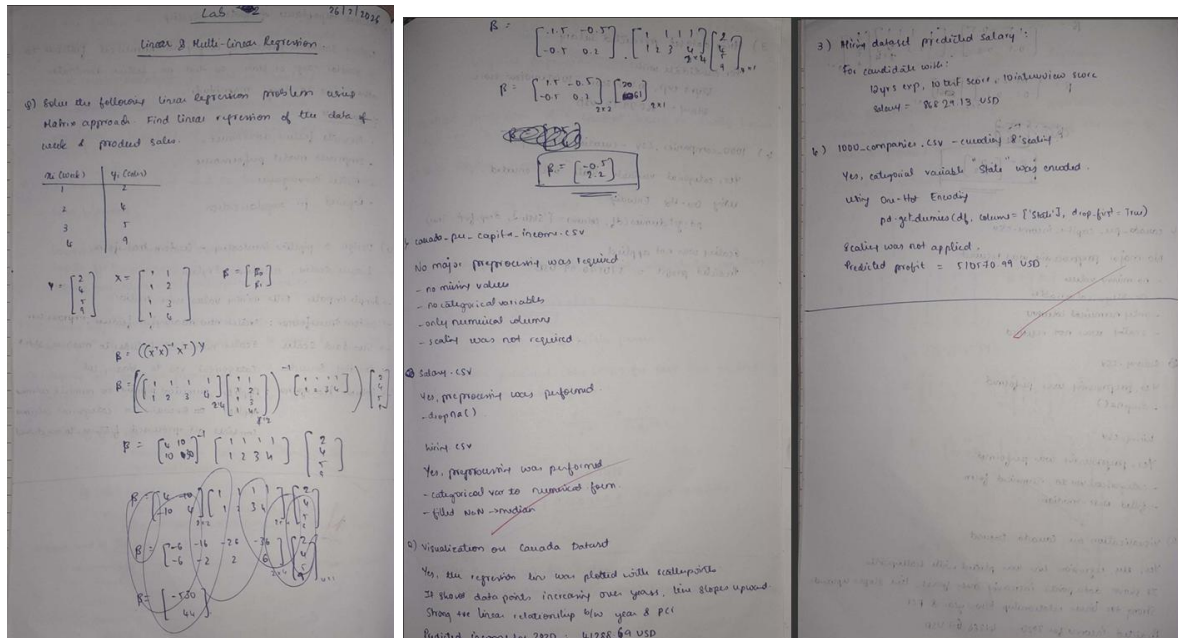
#Cons: Not suitable for non-normal data; may miss outliers in skewed distributions.

```
df_encoded_copy2['Salary_zscore'] = stats.zscore(df_encoded_copy2['Salary'])
df_encoded_copy2['Salary'] = np.where(df_encoded_copy2['Salary_zscore'].abs() > 3, np.nan,
df_encoded_copy2['Salary']) # Replace outliers with NaN
print(df_encoded_copy2.head())
#Removing Outliers
# Median replacement for outliers
#Pros: Keeps distribution shape intact, useful when capping isn't feasible.
#Cons: May distort data if outliers represent real phenomena.
df_encoded_copy3['Salary_zscore'] = stats.zscore(df_encoded_copy3['Salary'])
median_salary = df_encoded_copy3['Salary'].median()
df_encoded_copy3['Salary'] = np.where(df_encoded_copy3['Salary_zscore'].abs() > 3,
median_salary, df_encoded_copy3['Salary'])
print(df_encoded_copy3.head())
```

Program 3

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

Screenshot



Code:

```
# Predict Canada's Per Capita Income for 2020
```

```
# Import libraries
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.linear_model import LinearRegression
```

```
# Load dataset
```

```
df = pd.read_csv('canada_per_capita_income.csv')
```

```
# Display first 5 rows
```

```
print(df.head())
```

```
# Define independent and dependent variables
```

```
X = df[['year']] # Feature
```

```
y = df['per capita income (US$)']
```

```
# Create model
```

```
model = LinearRegression()
```

```
# Train model
```

```

model.fit(X, y)

# Predict income for year 2020
predicted_income = model.predict([[2020]])

print("Predicted Per Capita Income in 2020:", predicted_income[0])

plt.scatter(X, y)
plt.plot(X, model.predict(X))
plt.xlabel("Year")
plt.ylabel("Per Capita Income")
plt.show()
# -----
# Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
# -----
# Load dataset
df = pd.read_csv('salary.csv')

# Remove missing values
df.dropna(inplace=True)

# Define variables
X = df[['YearsExperience']]
y = df['Salary']

# Create and train model
model = LinearRegression()
model.fit(X, y)

# Predict salary for 12 years experience
predicted_salary = model.predict([[12]])

print("Predicted Salary for 12 years experience:", predicted_salary[0])

# ♦ Plot graph
plt.scatter(X, y)          # Actual data points
plt.plot(X, model.predict(X)) # Regression line

plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.title("Salary vs Experience (Linear Regression)")

plt.show()
# Import libraries

```

```

import pandas as pd
from sklearn.linear_model import LinearRegression
# -----
# Load dataset
df = pd.read_csv('hiring.csv')

# Convert word numbers to numeric
df['experience'] = df['experience'].replace({
    'zero':0,'one':1,'two':2,'three':3,'four':4,
    'five':5,'six':6,'seven':7,'eight':8,'nine':9,'ten':10
})

# Convert to numeric type
df['experience'] = pd.to_numeric(df['experience'], errors='coerce')

# Fill missing values using median
df['experience'] = df['experience'].fillna(df['experience'].median())
df['test_score(out of 10)'] = df['test_score(out of 10)'].fillna(
    df['test_score(out of 10)'].median()
)

X = df[['experience',
        'test_score(out of 10)',
        'interview_score(out of 10)']]

y = df['salary($)']

model = LinearRegression()
model.fit(X, y)

# Candidate 1
input1 = pd.DataFrame([[2, 9, 6]],
                       columns=X.columns)
salary1 = model.predict(input1)

# Candidate 2
input2 = pd.DataFrame([[12, 10, 10]],
                       columns=X.columns)
salary2 = model.predict(input2)

print("Predicted Salary (2 yr exp, 9 test, 6 interview):", salary1[0])
print("Predicted Salary (12 yr exp, 10 test, 10 interview):", salary2[0])
# 1000 Companies Profit Prediction
# Import libraries
import pandas as pd
from sklearn.linear_model import LinearRegression
# -----
# Load dataset

```

```

df = pd.read_csv('1000_Companies.csv')

# Handle categorical column (State)
df = pd.get_dummies(df, columns=['State'], drop_first=True)

# Define features and target
X = df.drop('Profit', axis=1)
y = df['Profit']

model = LinearRegression()
model.fit(X, y)

# Create input for Florida
# (Assuming columns are: R&D Spend, Administration, Marketing Spend,
# State_Florida, State_New York)

input_profit = pd.DataFrame([[91694.48,
                               515841.3,
                               11931.24,
                               1, 0]], # Florida = 1
                             columns=X.columns)

predicted_profit = model.predict(input_profit)

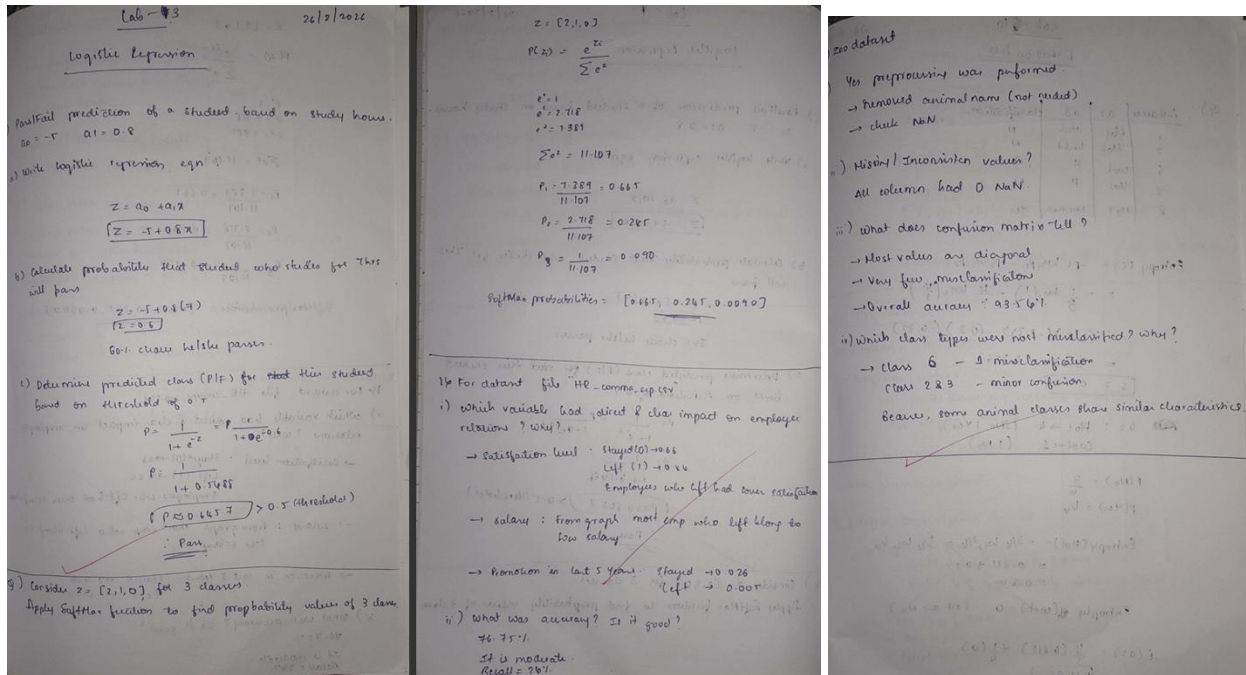
print("Predicted Profit:", predicted_profit[0])

```

Program 4

Build Logistic Regression Model for a given dataset

Screenshot



Code:

Logistic regression - binary classification

Import Libraries

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, classification_report

Load Dataset

df = pd.read_csv("HR_comma_sep.csv")

1. Exploratory Data Analysis

print("First 5 rows:")

print(df.head())

print("\nDataset Info:")


```

print(df.info())

print("\nEmployee Retention Count:")
print(df['left'].value_counts())

print("\nAverage values grouped by retention:")
print(df.groupby('left').mean(numeric_only=True))

# -----
# 2. Salary vs Retention (Bar Chart)
# -----

pd.crosstab(df.salary, df.left).plot(kind='bar')
plt.title("Salary vs Employee Retention")
plt.xlabel("Salary Level")
plt.ylabel("Number of Employees")
plt.show()

# -----
# 3. Department vs Retention (Bar Chart)
# -----

pd.crosstab(df.Department, df.left).plot(kind='bar', figsize=(10,5))
plt.title("Department vs Employee Retention")
plt.xlabel("Department")
plt.ylabel("Number of Employees")
plt.xticks(rotation=45)
plt.show()

# -----
# 4. Feature Selection
# -----
# Selecting important features
df2 = df[['satisfaction_level',
          'average_monthly_hours',
          'promotion_last_5years',
          'salary',
          'left']]

# Convert salary (categorical) to dummy variables
df2 = pd.get_dummies(df2, columns=['salary'], drop_first=True)

# -----
# 5. Train-Test Split
# -----

X = df2.drop('left', axis=1)
y = df2['left']

X_train, X_test, y_train, y_test = train_test_split(

```

```

X, y, test_size=0.3, random_state=42)

# -----
# 6. Build Logistic Regression Model
# -----

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# -----
# 7. Prediction & Accuracy
# -----

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:", accuracy)
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Logistic regression - multiclass classification
# Logistic Regression - Multiclass Classification (Zoo Dataset)

# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay

# -----
# 1. Load Dataset
# -----

zoo = pd.read_csv("zoo-data.csv")
class_type = pd.read_csv("zoo-class-type.csv")

print("First 5 rows:")
print(zoo.head())

# -----
# 2. Data Preprocessing
# -----

# Drop animal name (not useful for prediction)
zoo = zoo.drop("animal_name", axis=1)

```

```

# Check for null values
print("\nMissing values:")
print(zoo.isnull().sum())

# Features and Target
X = zoo.drop("class_type", axis=1)
y = zoo["class_type"]

# -----
# 3. Train-Test Split
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# -----
# 4. Build Logistic Regression Model
# -----

model = LogisticRegression(max_iter=1000, multi_class='multinomial')
model.fit(X_train, y_train)

# -----
# 5. Prediction
# -----

y_pred = model.predict(X_test)

# -----
# 6. Accuracy
# -----

accuracy = accuracy_score(y_test, y_pred)
print("\nModel Accuracy:", accuracy)

# -----
# 7. Confusion Matrix
# -----

cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.title("Confusion Matrix - Zoo Dataset")
plt.show()

```

Program 5

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshot

Lab - 5
26-2-2018
Decision Tree

Q) instance | a2 | a3 | classification

1	Hot	High	N
2	Hot	High	N
6	Cool	H	N
7	Hot	H	N
8	Hot	Normal	Yes

Entropy (S) = $-p_1 \log_2 p_1 - p_2 \log_2 p_2$
 $= -\frac{1}{5} \log_2 \left(\frac{1}{5}\right) - \frac{4}{5} \log_2 \left(\frac{4}{5}\right)$
 $= (0.2)(-2.32) - (0.8)(-0.32)$
 $= -0.464 + 0.256 = -0.208$
 $S = 0.72$

a2 : Hot → 4 (3 No 1 Yes)
Cool → 1 (1 No)

$P(\text{No}) = \frac{3}{5}$
 $P(\text{Yes}) = \frac{1}{5}$

Entropy (Hot) = $-\frac{3}{5} \log_2 \frac{3}{5} - \frac{1}{5} \log_2 \frac{1}{5}$
 $= 0.311 + 0.5 = 0.811$

Entropy (Cool) = 0 (all are No)

$E(a2) = \frac{4}{5}(0.811) + \frac{1}{5}(0)$
 $= 0.6488$

Info Gain (a2) = $0.72 - 0.6488$
 $= 0.071$

a3 : High → 4 (1 No)
Normal → 1 (1 Yes)

Entropy (High) = 0
 Entropy (Normal) = 0

$E(a3) = 0$

$I_G(a3) = 0.72 - 0$
 $I_G(a3) = 0.72$

Splitting node should be a3 ($I_G(a3) > I_G(a2)$)

Q) IRIS Dataset

→ Accuracy = 1.0 (100%)

→ Confusion matrix

	Actual \ Predicted	0	1	2
Predicted	0	5	0	0
	1	0	5	0
	2	0	0	5

all predictions are correctly classified
 off diagonal val are zero

→ No misclassification
 → No classes confused

Q) Petal consumption Dataset

→ Regression tree

split dataset based feature value
 each leaf node provide numerical value

→ Most imp feature!

1. Avg income.
2. Petrol tax.

→ Error matrix : RMSE = 131.71
 MAE = 98.3

Code:

Decision Tree - IRIS Dataset

Import Libraries

import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay

import matplotlib.pyplot as plt

Load Dataset

iris = pd.read_csv("iris.csv")

print("First 5 rows:")

print(iris.head())

```

# -----
# Features and Target
# -----
X = iris.drop("species", axis=1) # change column name if different
y = iris["species"]

# -----
# Train-Test Split (80-20)
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# -----
# Build Model
# -----
model = DecisionTreeClassifier()
model.fit(X_train, y_train)

# -----
# Prediction
# -----
y_pred = model.predict(X_test)

# -----
# Accuracy
# -----
accuracy = accuracy_score(y_test, y_pred)
print("\nIRIS Dataset Accuracy:", accuracy)

# -----
# Confusion Matrix
# -----
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.title("Confusion Matrix - IRIS Dataset")
plt.show()

```

Decision Tree - Drug Dataset

```

# Import Libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder

```

```

from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# -----
# Load Dataset
# -----
drug = pd.read_csv("drug.csv")

print("First 5 rows:")
print(drug.head())

# -----
# Data Preprocessing
# -----

# Encode categorical columns
le = LabelEncoder()

for column in drug.columns:
    if drug[column].dtype == 'object':
        drug[column] = le.fit_transform(drug[column])

# -----
# Features and Target
# -----
X = drug.drop("Drug", axis=1) # change if column name different
y = drug["Drug"]

# -----
# Train-Test Split (80-20)
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# -----
# Build Model
# -----
model = DecisionTreeClassifier()
model.fit(X_train, y_train)

# -----
# Prediction
# -----
y_pred = model.predict(X_test)

# -----
# Accuracy
# -----

```

```

accuracy = accuracy_score(y_test, y_pred)
print("\nDrug Dataset Accuracy:", accuracy)

# -----
# Confusion Matrix
# -----
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.title("Confusion Matrix - Drug Dataset")
plt.show()

```

Regression Tree - Petrol Consumption Dataset

```

# Import Libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error

# -----
# Load Dataset
# -----
data = pd.read_csv("petrol_consumption.csv")

print("First 5 rows:")
print(data.head())

# -----
# Features and Target
# -----
# Assuming last column is Petrol_Consumption
X = data.iloc[:, :-1] # All columns except last
y = data.iloc[:, -1] # Last column (target)

# -----
# Train-Test Split (80% - 20%)
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# -----
# Build Regression Tree Model
# -----
regressor = DecisionTreeRegressor(random_state=42)
regressor.fit(X_train, y_train)

```

```

# -----
# Prediction
# -----
y_pred = regressor.predict(X_test)

# -----
# Evaluation Metrics
# -----
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)

print("\nModel Evaluation Metrics:")
print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)

```


Program 6

Build KNN Classification model for a given dataset.

Screenshot

KNN Classification

Q) Consider given dataset for K=3 & test data (X, 35, 100) as (Person, Age, Salary) solve using KNN classifier model & predict the target.

Person	Age	Salary	Target
A	15	10	N
B	23	55	N
C	26	30	N
D	41	80	Y
E	43	70	Y
F	38	60	Y
X	35	100	?

Solu: Age, Salary = 100 → X
 $Dist = \sqrt{(Age - Age_i)^2 + (Salary - Salary_i)^2}$

Person	Age	Salary	Target	Dist
A	15	10	N	$\sqrt{(35-15)^2 + (100-10)^2} = 72.81$
B	23	55	N	$\sqrt{(35-23)^2 + (100-55)^2} = 46.73$
C	26	30	N	$\sqrt{(35-26)^2 + (100-30)^2} = 71.77$
D	41	80	Y	$\sqrt{(35-41)^2 + (100-80)^2} = 40.47$
E	43	70	Y	$\sqrt{(35-43)^2 + (100-70)^2} = 31.07$
F	38	60	Y	$\sqrt{(35-38)^2 + (100-60)^2} = 60.00$
X	35	100	?	

3 nearest neighbors: E, C, D
 Majority votes: Y
 Target (X, 35, 100) = Y

For Iris Dataset

How to choose the K value? Demonstrate using accuracy rate & error rate.

In KNN, the value of K (no. of neighbors) determines how the model classifies a new data point.

Method to choose K:

Optimal K is chosen by testing multiple K values, calculating accuracy rate, error rate.

Accuracy = $\frac{\text{Correct predictions}}{\text{Total predictions}}$

Error rate = $1 - \text{Accuracy}$

Example:

If accuracy = 1.0
 error rate = 0
 ⇒ model predicts all test samples properly.

2) Dataset Dataset

What is the purpose of feature scaling? How to perform it?

Feature scaling is used to normalize the range of independent variables so that all features contribute equally to the model. This is important for KNN because it uses distance calculations.

Feature scaling is performed by:

- Min-Max scaling: Rescales data to fixed range (0 to 1)
- Standardization (Z score): Transforms mean (μ) of 0 & SD of 1

$$X_{\text{scaled}} = \frac{X - X_{\text{min}}}{X_{\text{max}} - X_{\text{min}}}$$

$$Z = \frac{X - \mu}{\sigma}$$

Code:

```
# KNN Classifier for IRIS Dataset
# Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
iris = pd.read_csv("iris.csv")

# Features and target
X = iris.drop("species", axis=1) # change if target column name differs
y = iris["species"]

# Train-Test Split (80-20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# KNN Model
k = 5
```

```

model = KNeighborsClassifier(n_neighbors=k)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

# Classification Report
print("Classification Report:\n", classification_report(y_test, y_pred))

# Plot Confusion Matrix
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix - Iris Dataset")
plt.show()

# KNN Classifier for Diabetes Dataset (With Feature Scaling)
# Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Load dataset
diabetes = pd.read_csv("diabetes.csv")

# Features and target
X = diabetes.drop("Outcome", axis=1)
y = diabetes["Outcome"]

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# KNN Model
k = 7
model = KNeighborsClassifier(n_neighbors=k)

```

```

model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

# KNN Classifier for Heart Dataset (Finding Best K)
# Import libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset
heart = pd.read_csv("heart.csv")

# Features and target
X = heart.drop("target", axis=1)
y = heart["target"]

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Find best K
scores = []

for k in range(1,21):
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    scores.append(accuracy_score(y_test, pred))

# Plot K vs Accuracy

```

```

plt.plot(range(1,21), scores)
plt.xlabel("K value")
plt.ylabel("Accuracy")
plt.title("Finding Best K")
plt.show()

# Best K
best_k = scores.index(max(scores)) + 1
print("Best K:", best_k)

# Train final model
model = KNeighborsClassifier(n_neighbors=best_k)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

# Plot Confusion Matrix
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix - Heart Dataset")
plt.show()

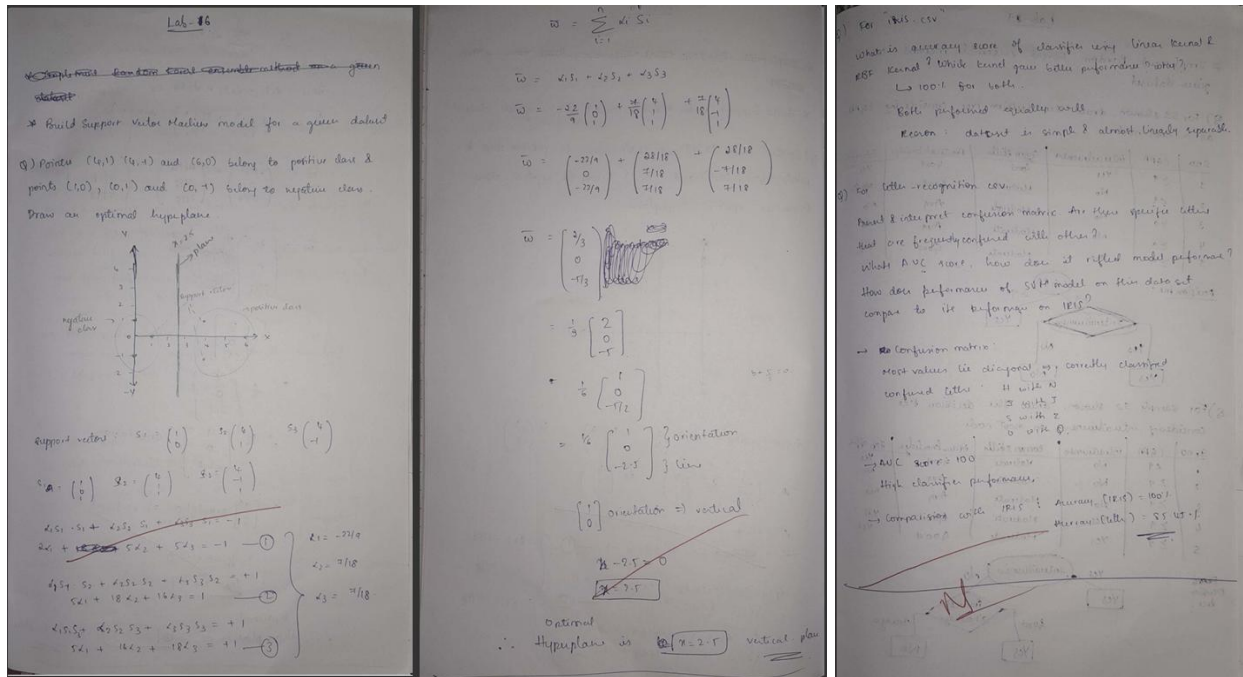
# Classification Report
print("Classification Report:\n", classification_report(y_test, y_pred))

```

Program 7

Build Support vector machine model for a given dataset

Screenshot



Code:

SVM Classifier for IRIS Dataset (Linear & RBF Kernel)

Import libraries

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, confusion_matrix

Load IRIS dataset

iris = pd.read_csv("iris.csv")

Display dataset

print(iris.head())

Features and target

X = iris.drop("species", axis=1)

y = iris["species"]

Train-test split (80% training, 20% testing)

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -----
# SVM using RBF kernel
# -----
rbf_model = SVC(kernel='rbf')

rbf_model.fit(X_train, y_train)

y_pred_rbf = rbf_model.predict(X_test)

print("RBF Kernel Accuracy:", accuracy_score(y_test, y_pred_rbf))

print("Confusion Matrix (RBF):")
print(confusion_matrix(y_test, y_pred_rbf))

# -----
# SVM using Linear kernel
# -----
linear_model = SVC(kernel='linear')

linear_model.fit(X_train, y_train)

y_pred_linear = linear_model.predict(X_test)

print("Linear Kernel Accuracy:", accuracy_score(y_test, y_pred_linear))

print("Confusion Matrix (Linear):")
print(confusion_matrix(y_test, y_pred_linear))
# SVM Classifier for Letter Recognition Dataset + ROC + AUC
# Import libraries
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc

# Load dataset
letter = pd.read_csv("letter-recognition.csv")

print(letter.head())

```

```

# Features and target
X = letter.iloc[:,1:] # features
y = letter.iloc[:,0]  # letter label

# Train-test split (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Train SVM model
model = SVC(kernel='linear', probability=True)
model.fit(X_train, y_train)
# Prediction
y_pred = model.predict(X_test)
# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
# Confusion Matrix
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# -----
# ROC Curve and AUC Score
# -----

# Convert labels to binary for ROC
classes = sorted(y.unique())
y_test_bin = label_binarize(y_test, classes=classes)

# Predict probabilities
y_score = model.predict_proba(X_test)

# ROC for first class
fpr, tpr, _ = roc_curve(y_test_bin[:,0], y_score[:,0])
roc_auc = auc(fpr, tpr)

# Plot ROC Curve
plt.figure()
plt.plot(fpr, tpr, label="ROC curve (AUC = %0.2f)" % roc_auc)
plt.plot([0,1], [0,1], linestyle='--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()

print("AUC Score:", roc_auc)

```

Program 8

Implement Random forest ensemble method on a given dataset.

Screenshot

Lab-8

10. Implement Random Forest ensemble method on given dataset.

Q) For S2 shown, draw the decision tree considering CGPA as a root node.

Sr no	CGPA	Intelligence	Comm skills	Practical knowledge	Job offer
1	≥ 9	Yes	Good	Good	Yes
2	< 9	No	Moderate	Avg	No
3	≥ 9	No	Moderate	Avg	No
4	≥ 9	No	Moderate	Good	Yes
5	≥ 9	Yes	Moderate	Good	Yes

Decision tree:

```

graph TD
    Root((CGPA)) -- "≥ 9" --> Yes1[Yes]
    Root -- "< 9" --> No1[No]
    Yes1 --> Yes1L[Yes]
    No1 --> No1L[No]
  
```

Q) For sample S2 shown, draw the decision tree considering intelligence as root node.

Sr no	CGPA	Intelligence	Comm skills	Practical knowledge	Job offer
1	≥ 9	No	Moderate	Good	Yes
2	≥ 9	No	Moderate	Avg	No
3	≥ 9	No	Moderate	Avg	No
4	≥ 9	Yes	Moderate	Good	Yes
5	≥ 9	Yes	Moderate	Good	Yes

Decision tree:

```

graph TD
    Root((Intelligence)) -- "Yes" --> Yes2[Yes]
    Root -- "No" --> No2[No]
    Yes2 --> Yes2L[Yes]
    No2 --> Knowledge{Practical Knowledge}
    Knowledge -- "Good" --> Yes2L
    Knowledge -- "Average" --> No2L[No]
  
```

Iris dataset:

Q) What is the best accuracy score and confusion matrix of the classifier you observed & using how many trees?

→ Best accuracy obtained is 1.0 (100%)

Accuracy with 10 trees = 0.966 (96.6%)

Best accuracy achieved using 1 tree.

Confusion matrix (for best model):

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

(for default 10 trees):

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Code:

```

# Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load dataset
data = pd.read_csv("iris.csv")

# Features and target
X = data.drop("species", axis=1) # assuming target column is named 'species'
y = data["species"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=68
)

# -----
  
```



```

# 1. Default Random Forest (n_estimators = 10)
# -----
rf_default = RandomForestClassifier(n_estimators=10, random_state=42)
rf_default.fit(X_train, y_train)

y_pred_default = rf_default.predict(X_test)
default_score = accuracy_score(y_test, y_pred_default)

print("Accuracy with default (10 trees):", default_score)

# -----
# 2. Hyperparameter tuning (number of trees)
# -----
best_score = 0
best_n = 0

for n in range(1, 101): # try 1 to 100 trees
    rf = RandomForestClassifier(n_estimators=n, random_state=42)
    rf.fit(X_train, y_train)

    y_pred = rf.predict(X_test)
    score = accuracy_score(y_test, y_pred)

    if score > best_score:
        best_score = score
        best_n = n

print("Best accuracy:", best_score)
print("Best number of trees:", best_n)
from sklearn.metrics import confusion_matrix

# For default model
cm_default = confusion_matrix(y_test, y_pred_default)
print("Confusion Matrix (Default - 10 trees):\n", cm_default)

# For best model (inside loop OR after finding best_n)
rf_best = RandomForestClassifier(n_estimators=best_n, random_state=42)
rf_best.fit(X_train, y_train)

y_pred_best = rf_best.predict(X_test)
cm_best = confusion_matrix(y_test, y_pred_best)

print("\nConfusion Matrix (Best Model):\n", cm_best)

```

Program 9

Implement Boosting ensemble method on a given dataset.

Screenshot

Implemented AdaBoost ensemble method:

Q) Consider AdaBoost algorithm. For the following sample data, show the Decision stump calculation steps for the attribute CPM.

CPM	Interactions	Radical banding	Leavesville	Feb
> 9	Yes	Good	Good	Yes
< 9	No	Good	moderate	Yes
> 9	No	Any	moderate	No
< 9	No	Good May	good	No
> 9	Yes	Good	moderate	Yes
> 9	Yes	Good	moderate	Yes

Step 1: Random bootstrap sample:

CPM > 9 → Yes
 < 9 → No

Step 2: $E_1 = \frac{1}{n} \sum_{i=1}^n W_1(x_i) \cdot d(x_i)$

where $W_1(x_i) = 0$ if prediction correct, $W_1(x_i) = 1$ if prediction wrong

CPM	predicted class	Actual class	Weight
> 9	Yes	Yes	1/6
< 9	No	Yes	1/6
< 9	Yes	No	1/6
< 9	No	No	1/6
> 9	Yes	Yes	1/6
> 9	Yes	Yes	1/6

$E_{\text{CPM}} = \left(\frac{1}{6}\right) \left(\frac{1}{6}\right) + \left(\frac{1}{6}\right) \left(\frac{1}{6}\right) + \frac{1}{3} = 0.333$

Step 3: Calculate weight of weak classifier

$W_{\text{CPM}} = \frac{1}{2} \ln \left(\frac{1 - E_{\text{CPM}}}{E_{\text{CPM}}} \right) = \frac{1}{2} \times \ln \left(\frac{1 - 0.333}{0.333} \right) = 0.347$

Step 4: Calculate normalizing factor

$Z_{\text{CPM}} = \left(\frac{1}{6} \times e^0\right) + \left(\frac{1}{6} \times e^{0.347}\right)$

≈ 0.9428

Step ① update weights:

$$w(d_{ij})_{\text{new}} = \frac{w(d_{ij})_{\text{old}} + \text{current instance} \cdot e}{Z_{\text{class}}}$$

$$= \frac{(V_6) e^{-0.317}}{0.7628} = 0.1249 \leftarrow \text{current instances}$$

$$w(d_{ij})_{\text{new}} = \frac{w(d_{ij})_{\text{old}} + \text{current instance} \cdot e}{Z_{\text{class}}}$$

$$= \frac{(V_6) e^{-0.317}}{0.9438} = 0.2501 \leftarrow \text{current instances}$$

class	predicted	actual	weight
yes	Yes	Yes	0.1249
yes	No	Yes	0.2501
yes	No	No	0.2501
yes	No	No	0.1249
no	Yes	Yes	0.1249
no	Yes	No	0.1249

✳️ Inverse CSV

What is the best accuracy score & confusion matrix of classifier & how many errors?

→ accuracy with 10 estimators: 91.82%

Best accuracy is 93.40% → 13 estimators

Confusion matrix

	Actual Yes	Actual No
Predicted Yes	10	11
Predicted No	7	12

Code:

```
# Import libraries
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import AdaBoostClassifier
```

```
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
# Load dataset
```

```
data = pd.read_csv("income.csv")
```

Features and target

```
X = data.drop("income_level", axis=1)
```

```
y = data["income_level"]
```

Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(X,
```

```
X, y, test_size=0.2, random_state=42
```

)

1. AdaBoost with 10 estimators

```

ada_default = AdaBoostClassifier(n_estimators=10, random_state=42)
ada_default.fit(X_train, y_train)

y_pred_default = ada_default.predict(X_test)
default_score = accuracy_score(y_test, y_pred_default)

print("Accuracy with 10 estimators:", default_score)

cm_default = confusion_matrix(y_test, y_pred_default)
print("Confusion Matrix (Default):\n", cm_default)

# -----
# 2. Hyperparameter tuning
# -----
best_score = 0
best_n = 0
best_cm = None

for n in range(1, 101):
    ada = AdaBoostClassifier(n_estimators=n, random_state=42)
    ada.fit(X_train, y_train)

    y_pred = ada.predict(X_test)
    score = accuracy_score(y_test, y_pred)

    if score > best_score:
        best_score = score
        best_n = n
        best_cm = confusion_matrix(y_test, y_pred)

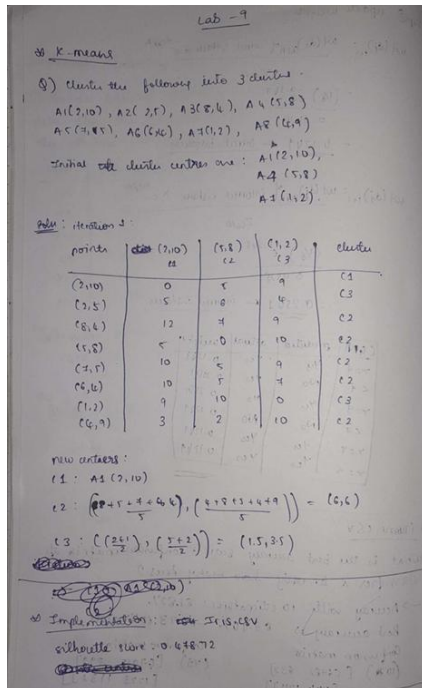
print("\nBest Accuracy:", best_score)
print("Best number of estimators:", best_n)
print("Confusion Matrix (Best Model):\n", best_cm)

```

Program 10

Build k-Means algorithm to cluster a set of data stored in a .CSV file.

Screenshot



Code:

```
# Import libraries
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# Load dataset
data = pd.read_csv("iris.csv")

# -----
# Preprocessing
# -----

# Drop target column if present (unsupervised learning)
if "species" in data.columns:
    X = data.drop("species", axis=1)
else:
    X = data

# Optional: Feature scaling (recommended for K-Means)
scaler = StandardScaler()
```

```

X_scaled = scaler.fit_transform(X)

# -----
# 1. Apply K-Means (k = 3)
# -----
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X_scaled)

labels = kmeans.labels_

print("Cluster labels:\n", labels)

# -----
# 2. Evaluate clustering
# -----
score = silhouette_score(X_scaled, labels)
print("Silhouette Score:", score)

# -----
# 3. Cluster centers
# -----
print("Cluster Centers:\n", kmeans.cluster_centers_)

```

Program 11

Implement Dimensionality reduction using Principal Component Analysis (PCA) method.

Screenshot

Lab-10

PCA

10 Implement dimensionality reduction using PCA method.

Q) In given table, reduce dimension from 2 to 1 using PCA. Compute 1st principal component.

Feature	Example 1	Example 2	Example 3	Example 4
x ₁	4	8	13	7
x ₂	11	4	5	16

Given: Eigen values: $\lambda_1 = 30.3849$, $\lambda_2 = 6.6151$
 Eigen vectors: $e_1 = \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix}$, $e_2 = \begin{bmatrix} 0.8303 \\ 0.5574 \end{bmatrix}$

Soln:

Calculate mean: $\bar{x}_1 = \frac{4+8+13+7}{4} = 8$, $\bar{x}_2 = \frac{11+4+5+16}{4} = 8.5$

Covariance matrix:

$$\text{Cov}(x_1, x_1) = \frac{1}{N-1} \sum_{k=1}^N (x_{1k} - \bar{x}_1)(x_{1k} - \bar{x}_1)$$

$$= \frac{1}{3} ((4-8)^2 + (8-8)^2 + (13-8)^2 + (7-8)^2)$$

$$= 14$$

$$\text{Cov}(x_1, x_2) = \frac{1}{N-1} \sum_{k=1}^N (x_{1k} - \bar{x}_1)(x_{2k} - \bar{x}_2)$$

$$= \frac{1}{3} ((4-8)(11-8.5) + (8-8)(4-8.5) + (13-8)(5-8.5) + (7-8)(16-8.5))$$

$$= -11$$

$$\text{Cov}(x_2, x_1) = -11$$

$$\text{Cov}(x_2, x_2) = \frac{1}{3} ((11-8.5)^2 + (4-8.5)^2 + (5-8.5)^2 + (16-8.5)^2)$$

$$= 23$$

$$S = \begin{bmatrix} \text{Cov}(x_1, x_1) & \text{Cov}(x_1, x_2) \\ \text{Cov}(x_2, x_1) & \text{Cov}(x_2, x_2) \end{bmatrix} = \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$

eigen values:
 $\lambda_1 = 30.3849$
 $\lambda_2 = 6.6151$

eigen vector:
 $e_1 = \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix}$
 $e_2 = \begin{bmatrix} 0.8303 \\ 0.5574 \end{bmatrix}$

first principal component:

$$e_1^T \begin{bmatrix} x_{1k} - \bar{x}_1 \\ x_{2k} - \bar{x}_2 \end{bmatrix} = \begin{bmatrix} 0.5574 & -0.8303 \end{bmatrix} \begin{bmatrix} x_{1k} - \bar{x}_1 \\ x_{2k} - \bar{x}_2 \end{bmatrix}$$

$$= 0.5574(4-8) - 0.8303(11-8.5)$$

$$= -4.3053$$

Feature	Ev 1	Ev 2	Ev 3	Ev 4
x ₁	4	8	13	7
x ₂	11	4	5	16
first principal component	-4.3053	3.7361	5.6929	-5.1238

10. Heatmap

Report accuracy scores before & after applying PCA

Before PCA	After PCA
SVM: 86.4%	SVM: 85.86%
logistic regression: 85.32%	LR: 85.326%
Random forest: 87.5%	Random forest: 85.87%
Best accuracy: 87.5%	Best: 85.87%

Code:

```
# Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score
from sklearn.decomposition import PCA

# Models
from sklearn.svm import SVC
```

```

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

# Load dataset
data = pd.read_csv("heart.csv")

# -----
# Features and target
# -----
X = data.drop("HeartDisease", axis=1)
y = data["HeartDisease"]

# -----
# Column separation
# -----
categorical_cols = ["Sex", "ChestPainType", "RestingECG", "ExerciseAngina", "ST_Slope"]
numeric_cols = ["Age", "RestingBP", "Cholesterol", "FastingBS", "MaxHR", "Oldpeak"]

# -----
# Preprocessing
# -----
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_cols),
        ("cat", OneHotEncoder(), categorical_cols)
    ]
)

# -----
# Train-test split
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -----
# Models
# -----
models = {
    "SVM": SVC(),
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Random Forest": RandomForestClassifier()
}

# -----
# BEFORE PCA
# -----
print("---- BEFORE PCA ----")

```

```

best_score = 0

for name, model in models.items():
    pipe = Pipeline([
        ("preprocess", preprocessor),
        ("classifier", model)
    ])

    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)
    score = accuracy_score(y_test, y_pred)

    print(f'{name}: {score}')

    if score > best_score:
        best_score = score

print("Best Accuracy before PCA:", best_score)

# -----
# AFTER PCA
# -----
print("\n---- AFTER PCA ----")

pca = PCA(n_components=0.95)

best_score_pca = 0

for name, model in models.items():
    pipe = Pipeline([
        ("preprocess", preprocessor),
        ("pca", pca),
        ("classifier", model)
    ])

    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)
    score = accuracy_score(y_test, y_pred)

    print(f'{name}: {score}')

    if score > best_score_pca:
        best_score_pca = score

print("Best Accuracy after PCA:", best_score_pca)

```